title: "Formal Audit Report: API Design Document v1.1" date: 2026-04-04 type: audit auditor: Claude Opus 4.6 (Anthropic) subject: "API Design: REST, GraphQL, SDKs, and Public Interfaces — v1.1" disclaimer: > DISCLAIMER: No information within this document should be taken for granted. Any statement or premise not backed by a real logical definition or verifiable reference may be invalid, erroneous, or a hallucination. The reader is responsible for independently verifying all claims. This audit was conducted via web research, direct source verification, and full reading of both the subject document and its cited PALS's Law source (v1.5.4). Verification depth varies by claim — limitations are noted where applicable.

Formal Audit Report: API Design Document v1.1

0. Executive Summary

The document is **substantially correct and analytically strong**. Of approximately 40 verifiable factual claims, 35 check out cleanly against primary sources, 3 require minor corrections, and 2 are imprecise but defensible. The PALS's Law integration — the document's most significant structural addition in v1.1 — is well-executed: the source material is a rigorous engineering formalization with real theoretical backing, and its adaptation to API design contexts is both honest and analytically valuable. The document's most important remaining gaps are topical (event-driven patterns, long-running operations, API governance) rather than factual.

Corrections required: 3 (minor factual) **Corrections recommended: 4** (precision, terminology, completeness) **Structural assessment: strong**

---

1. Methodology

This audit was conducted in three phases:

1. **Claim-by-claim verification** of all factual assertions against primary sources (RFCs, official documentation, original publications, specification repositories). Web search was used to verify URLs, dates, authorship, and technical details.
2. **Source document review** of PALS's Law v1.5.4 (12-page PDF, full formal specification with definitions, proofs, empirical references, taxonomy, limitations, and practitioner artifacts).
3. **Completeness assessment** against the current state of API design literature, industry practice, and 2025–2026 developments.

---

2. Factual Accuracy — Claim-by-Claim

2.1 REST Foundations (Sections 2.1–2.3)

Fielding's dissertation. Title, institution (UC Irvine), year (2000), and URL are correct. The constraint derivation chain (Client-Server → Stateless → Cache → Uniform Interface → Layered System → Code-on-Demand) exactly matches Sections 5.1.2–5.1.7 of the dissertation. The four Uniform Interface sub-constraints are confirmed verbatim from Section 5.1.5. **Verdict: correct.**

Fielding's authorship. The v1.1 document describes Fielding as "co-author of HTTP/1.0 (RFC 1945) and lead author of the HTTP/1.1 specifications (RFC 2068, RFC 2616)." RFC 1945 lists Berners-Lee, Fielding, and Frystyk — co-authorship is confirmed. For HTTP/1.1, Fielding is first-listed among seven co-authors on RFC 2616 and subsequently co-editor of the RFC 7230–7235 and RFC 9110–9112 revision series. "Lead author" is an informal characterization; "first-listed co-author and editor" would be more precise. **Verdict: defensible but imprecise.**

Recommendation: Consider "first-listed co-author and editor" if maximum precision is desired. Current phrasing is not wrong.

Fielding's 2008 blog post. Title, URL, date (October 20, 2008), and content are confirmed. The post exists at `roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven` and contains the quoted position that APIs must be hypertext-driven to be RESTful. **Verdict: correct.**

Richardson Maturity Model. Leonard Richardson presented the model at QCon San Francisco on November 19, 2008. Martin Fowler published his popularization article on March 18, 2010. The four levels (0–3) are accurately described. One nuance: the level names ("Swamp of POX," etc.) are Fowler's terminology, not Richardson's original labels. **Verdict: correct.**

HTTP method semantics table. All entries verified against RFC 9110 Section 9.2.2. PATCH is correctly described as not guaranteed idempotent per RFC 5789. DELETE is correctly listed as idempotent. The footnote about PATCH is accurate. **Verdict: correct.**

2.2 Idempotency (Section 2.4)

Stripe idempotency keys. The v1.1 document says Stripe stores keys for "a bounded retention window." Stripe's official documentation states unambiguously: keys are removed after at least 24 hours. The v1.0 document correctly specified 24 hours; v1.1 regressed to a vaguer phrasing. **Verdict: technically not wrong, but a precision regression.**

Correction required: Restore the specific figure: "Stripe stores keys for at least 24 hours."

DELETE idempotency nuance. The v1.1 document correctly adds the explanation that DELETE idempotency concerns server state, not response codes — a first DELETE may return 200, a replay may return 404, but the resource is absent in both cases. This is accurate per RFC 9110. **Verdict: correct and improved from v1.0.**

2.3 Versioning (Section 2.5)

Stripe's versioning model. The v1.1 document correctly states Stripe "pins each account to the most recent API version available at the time of the account's first API request." This was corrected from v1.0's incorrect "pins each API key." The `Stripe-Version` header override and backward compatibility since 2011 are confirmed from Stripe's 2017 engineering blog post. **Verdict: correct.**

2.4 GraphQL (Sections 3.1–3.5)

Origin. Facebook internal development in 2012, open-sourced in 2015 — confirmed. First spec working draft July 2015, public announcement September 14, 2015. **Verdict: correct.**

Apollo Federation. Introduced May 30, 2019 — confirmed. **Verdict: correct.**

Composite Schemas Working Group. The v1.1 document still says "Composite Schema Working Group" (singular). The correct name is "Composite Schemas Working Group" (plural), and it is a subcommittee of the GraphQL Specification Working Group, not a standalone entity. **Verdict: incorrect — same error as v1.0.**

Correction required: Change "Composite Schema Working Group" to "Composite Schemas Working Group" (plural). The group is a subcommittee maintaining the specification at `graphql.github.io/composite-schemas-spec/`.

GraphQL Cursor Connections Specification. The v1.1 document now correctly uses the official name and parenthetically notes it is "commonly called the Relay pagination spec." The structural description (edges, nodes, pageInfo, cursor) is accurate per the specification at `relay.dev/graphql/connections.htm`. **Verdict: correct — improved from v1.0.**

2.5 SDK Design (Sections 5.1–5.4)

SDK design principles attribution. The v1.1 document correctly attributes the five principles (idiomatic, consistent, approachable, diagnosable, dependable) to "Microsoft Azure SDK Design Guidelines" with the correct URL (`azure.github.io/azure-sdk/general_introduction.html`). This was the most significant factual error in v1.0 (misattributed to Auth0 and IBM Watson) and is now fixed. **Verdict: correct — major improvement from v1.0.**

Pragmatic Engineer SDK generators claim. Confirmed. The article by Gergely Orosz and Quentin Pradet (2025) states the "one engineer, 4–5 SDKs" figure. Tools listed (OpenAPI Generator, Speakeasy, Stainless, Smithy, TypeSpec) are all real, though Smithy and TypeSpec are more precisely API modeling tools that feed into code generation pipelines. **Verdict: correct.**

2.6 Public Interface Design (Section 6)

Stripe's design culture. The v1.1 document correctly recharacterizes this as "it is reportedly not unusual to circulate 20-page design documents proposing individual API changes" — accurately

reflecting the source (CJ Avilla's talk summarized by Postman), rather than v1.0's implication of a single canonical design document. **Verdict: correct — improved from v1.0.**

Stripe prefixed IDs and key prefixes. `ch_`, `cus_`, `pi_`, `sub_` confirmed. `sk_test_` / `sk_live_` confirmed. **Verdict: correct.**

RFC 9457. Correctly identified as successor to RFC 7807, published July 2023. The five members (`type`, `title`, `status`, `detail`, `instance`) are accurately listed. The characterization as defining `application/problem+json` is correct. **Verdict: correct.**

RFC 9700. Correctly identified as "Best Current Practice for OAuth 2.0 Security" (BCP 240), published January 2025. Deprecation of Implicit Grant and ROPC with "MUST NOT" language is confirmed. **Verdict: correct.**

OWASP API Security Top 10. The 2023 edition is correctly identified as the most recent. Published June–July 2023. No newer edition exists as of April 2026. **Verdict: correct.**

2.7 Specification Formats (Section 7)

The v1.1 document uses "OpenAPI 3.x" and "AsyncAPI 3.x" — deliberately broader than v1.0's specific version numbers. OpenAPI 3.2.0 was released September 2025; AsyncAPI 3.1.0 is the current release. The "3.x" framing avoids the version-precision issue flagged in the v1.0 audit. **Verdict: acceptable — trades precision for durability.**

2.8 Error Taxonomy Remaining Correction

Correction required: Line 217 — "Composite Schema Working Group" → "Composite Schemas Working Group."

---

3. PALS's Law Integration — Substantive Assessment

3.1 Source Document Quality

PALS's Law v1.5.4 is a 12-page formal specification with the following structure: informal statement, formal statement (two forms — operative and existential), empirical support (8 peer-reviewed references), theoretical foundations (4 references including STOC 2024), a 9-class error taxonomy, argument sketches, 6 explicit limitations, 5 architectural corollaries, and practitioner artifacts.

The formal structure is well-constructed. The separation between the operative form (§3.2 — empirical claim that $\delta > 0$ on realistic distributions) and the existential form (§3.3 — formally provable from finite-parameter and computability arguments) is the right architectural decision. The operative form carries the engineering weight; the existential form provides theoretical grounding. The document is explicit about this distinction.

The empirical references are real and correctly characterized. Kalai & Vempala (STOC 2024) proves a statistical lower bound on hallucination for calibrated language models. Xu, Jain & Kankanhalli (2024) establishes inevitability via computability-theoretic diagonalization. The empirical papers (Ji et al. 2023, Maynez et al. 2020, TruthfulQA, Kadavath et al. 2022, Perez et al. 2022, Sharma et al. 2023, Berglund et al. 2023, Zhou et al. 2023/IFEval) collectively ground five of nine error classes. The inclusion of Suzuki et al. (2025) — which proves the complementary result that hallucination-triggering inputs can be made statistically negligible — demonstrates honest engagement with counterarguments rather than cherry-picking.

The error taxonomy (§5) is practically valuable and well-scoped. Nine classes, each requiring distinct detection strategies, with explicit scope notes on adversarial contexts, policy violations, and multimodal extensions. The distinction between ERR_HALLUCINATION (fabricated facts) and ERR_REASONING (correct facts, invalid composition) is particularly useful and absent from most error taxonomies in the literature.

The limitations section (§7) is unusually rigorous for an engineering principle document. Six explicit limitations, each correctly scoped: non-computability of the error predicate, task/model/distribution dependence of δ, approximate independence in pipeline compounding, unresolved verification coverage vs. depth, Boolean predicate simplification, and the computability-theoretic vs. practical tension.

The document's self-positioning is honest and accurate. The Preamble explicitly states: "The constituent insights are established; their packaging as an enforceable engineering contract is the contribution." It places itself in the tradition of eponymous engineering principles (Hyrum's Law, Postel's Law) and does not claim theoretical novelty over the cited results. This is a legitimate form of contribution — making implicit, widely violated assumptions explicit and enforceable.

3.2 Quality of the Adaptation in the API Document

The API document imports four PALS definitions into the API design context:

1. **Untrusted output** → any model-produced request, parameter selection, tool call, or interpretation of API response must be assumed potentially wrong until verified.
2. **Verification boundary** → the layer at which the system checks whether a proposed API interaction is valid, safe, and committed (schema validation, auth checks, idempotency, retry classification, post-call state inspection).
3. **Architectural omission** → if a workflow lets an LLM trigger or interpret API actions without a declared verification step, the defect is in the system design.
4. **Error taxonomy** → failures partitioned into machine-meaningful classes so downstream automation does not guess whether to retry, revise, abandon, or escalate.

These adaptations are **well-executed**. They correctly map the general LLM-output-verification framework to the specific concerns of API design. The document does not overextend the source material or claim more than the source supports. The framing — "imported PALS definitions, adapted to API and tool contexts" — is accurate.

3.3 PALS-Derived Content Value

The PALS integration produces five concrete contributions that go beyond standard API design literature:

Verification-first design (§1.2–1.3). The argument that design-first vs. code-first is insufficient for machine consumers — that you need a third axis evaluating whether the contract is mechanically verifiable — is a genuinely useful reframing not present in the standard references (Microsoft, Stripe, Postman, Apollo). It derives a design requirement from a constraint rather than stating a preference.

Observable write state (§2.4). The extension to idempotency — that replay safety alone is insufficient if the caller cannot determine whether a write was proposed, accepted, committed, or partially applied — fills a gap in standard API design treatments. Stripe's own "Build on Stripe with LLMs" documentation implicitly validates this by providing operation-state inspection endpoints specifically for agent consumers.

Error taxonomy for machine recovery (§6.2). The five-class error taxonomy with explicit recovery semantics (retry-after-modification, retry-after-credential-change, refresh-state-first, backoff, assume-unknown) goes meaningfully beyond RFC 9457, which provides a structured envelope but not recovery guidance. This is the most practically useful new section.

Decision framework rewrite (§9). The addition of three questions absent from standard guides — What is the retry/replay model? How is failure classified? What is the verification boundary? — addresses architectural constraints rather than stylistic preferences. The reframing from "which is better" to "which contract can be verified under the actual failure modes of the system that will consume it" is sharper than the standard treatment in the cited references.

PALS-Aligned Checklist (§9.1). Six binary questions mapping to concrete implementation decisions: schema-validatable requests/responses, machine-readable failure classes, replay-safe mutations, observable commit state, bounded destructive actions, stable identifiers with state inspection. This is an actionable design audit tool that stands on its own merit regardless of the PALS label.

3.4 Substantive Criticisms of the PALS Source

These are honest gaps in the source material that propagate into the API document's framing:

Verification cost model. PALS's Law prescribes verification as a first-class concern but acknowledges (§5 scope note) that verification cost may exceed generation cost for some error classes. The API document inherits this gap. Practitioners need guidance on verification triage —

e.g., full automated verification for ERR_SCHEMA (cheap), sampling-based verification for ERR_HALLUCINATION (expensive, requires external ground truth). Neither document provides this.

Unquantified δ. The operative form asserts "non-negligible" but deliberately does not provide bounds. For API design decisions, engineers need order-of-magnitude guidance to calibrate verification investment. A mapping from error classes to observed δ ranges from the cited benchmarks would strengthen practical utility.

Pipeline correlation structures. The independence caveat (§7.3) correctly notes the product formula breaks down under shared context, but common correlation patterns in agentic API pipelines (cascade, masking, amplification) could be sketched without claiming completeness.

Adversarial boundary. In agentic API deployments, intrinsic error (ERR_HALLUCINATION producing a plausible-looking URL) and adversarial exploitation (prompt injection producing a malicious URL) are indistinguishable at the verification boundary. The PALS source correctly scopes prompt injection as extrinsic (§5 scope note), but the API document's verification-boundary concept may need to account for this indistinguishability in practice.

---

4. Completeness Assessment

4.1 Topics Well-Covered

The document provides strong, verified coverage of: REST foundations and Fielding's actual theory, the Richardson Maturity Model, HTTP method semantics, idempotency patterns (with the AI-agent extension), versioning strategies (including Stripe's rolling model), GraphQL schema design and federation, gRPC positioning, SDK design principles and generation tradeoffs, public API design patterns, error design (significantly expanded in v1.1), rate limiting, authentication and security (RFC 9700, OWASP), specification formats, and the AI-agent design axis.

4.2 Topics Still Missing

The following topics were flagged in the v1.0 audit as critical omissions. The v1.1 revision partially addresses some through the AI-framing sections but does not provide dedicated treatment:

Webhooks and event-driven API patterns. AsyncAPI is mentioned in the specification formats table but there is no dedicated section on webhook design, event payload contracts, delivery guarantees, retry semantics, or subscription management. These are a primary architectural pattern in production API design and cannot be substituted by the AI-agent framing.

Long-running operations. The document mentions observable write state and operation identifiers (through the PALS integration) but does not cover the standard patterns: polling,

webhook callbacks, the async request-acknowledge-poll pattern, or operation status resources. These are universal API design challenges with established conventions.

API governance. Design review processes, style guides, automated linting (Spectral, Redocly), shift-left enforcement in CI/CD, and organizational API lifecycle management are absent. The PALS checklist (§9.1) partially addresses the design-review angle but does not cover governance infrastructure.

Contract testing. Pact, Dredd, Schemathesis, and similar tools for verifying that implementations conform to their API contracts are unmentioned. This is particularly relevant given the document's emphasis on contracts and verification.

Batch and bulk operations. Standard patterns for multi-resource operations (bulk create, batch update, partial failure handling) are absent.

Hypermedia formats. HAL, JSON:API, JSON-LD, and Siren are not discussed. Given the document's treatment of HATEOAS as a theoretical concern, a brief note on the practical hypermedia format landscape would close the gap.

Google's Agent2Agent (A2A) protocol. Launched April 2025 with 50+ partners, donated to the Linux Foundation June 2025. The document mentions MCP but not A2A, which addresses the complementary concern of agent-to-agent communication (vs. MCP's agent-to-tool focus).

4.3 Assessment

The v1.1 revision significantly improved completeness on security (RFC 9700, OWASP), error design (RFC 9457, recovery taxonomy), and the AI-agent design axis. The remaining gaps are topical rather than analytical — the document's framework is sound, but its scope does not yet cover the full surface area of production API design in 2026.

---

5. Structural and Reference Quality

5.1 Reference Quality

The v1.1 reference list is cleaner and better-tiered than v1.0. The addition of primary standards (RFC 9110, RFC 9457, RFC 9700, OWASP) alongside the Microsoft Azure SDK Guidelines strengthens the citation foundation. The reference note distinguishing primary standards from secondary commentary is a good practice.

The PALS's Law reference (local file path + Zenodo DOI) is transparent about its provenance. The Zenodo DOI (`zenodo.org/records/19401530`) could not be resolved during this audit — it may not yet be indexed or may have been deposited very recently. This does not affect the analytical validity of the source material, which was reviewed in full, but readers attempting to follow the reference may encounter a dead link. The local file path (`core/PALS_LAW-v1.5.4.md`) is only useful to the document's author.

Recommendation: Ensure the Zenodo record is publicly accessible. Consider providing a secondary access path (e.g., a public repository) until the DOI resolves reliably.

5.2 Structural Notes

The v1.1 document has minor formatting inconsistencies: some section transitions lack horizontal rules (`---`) that separate major sections in the rest of the document (e.g., between §6.4 and §6.5, between §7 and §8). Section numbering is consistent but the transition from §1.3 (PALS) to §2 (REST) lacks a separator. These are cosmetic and do not affect content.

---

6. Corrections Summary

Required Corrections (3)

#	Location	Current Text	Correction	Severity
1	§2.4	"a bounded retention window"	"at least 24 hours" (per Stripe official docs)	Minor — precision regression
2	§3.3	"Composite Schema Working Group"	"Composite Schemas Working Group" (plural)	Minor — naming error, unfixed from v1.0
3	§References	Zenodo `records/19401530`	Verify DOI resolves publicly; add fallback URL	Minor — accessibility

Recommended Improvements (4)

#	Location	Recommendation	Rationale
1	§2.1	Consider "first-listed co-author and editor" for Fielding	More precise than "lead author," though current phrasing is defensible
2	Throughout	Consider adopting "agentic AI" or "AI-agent" alongside or instead of "AI-heavy systems"	"AI-heavy" is not a recognized term of art; industry uses "agentic AI," "AI-ready," "AI-native"
3	§8 / new section	Add coverage of webhooks, event-driven patterns, and long-running operations	Critical production API patterns absent from scope
4	§8 / new section	Mention Google's A2A protocol alongside MCP	Complementary protocol for agent-to-agent communication, relevant to the

#	Location	Recommendation	Rationale
			document's agentic framing

7. Final Verdict

The document is correct, analytically strong, and well-sourced. Its factual foundation — REST theory, GraphQL ecosystem, Stripe patterns, HTTP semantics, RFCs, OWASP — is verified against primary sources with only minor corrections needed. Its most significant addition in v1.1 — the PALS's Law integration — is based on a rigorous source document with real theoretical backing, honest limitations, and practical utility. The adaptation of PALS concepts to API design is well-executed and produces genuinely original analytical contributions (verification-first design, observable write state, machine-recovery error taxonomy, the PALS-Aligned Checklist) that go beyond what the standard API design literature covers.

The document's scope does not yet cover the full surface area of production API design — event-driven patterns, long-running operations, and API governance are the most important gaps. But within its stated scope, the document is reliable, well-reasoned, and honest about its own limitations.